Event-Driven Architecture (EDA) Pattern

Category: Data / messaging **One-liner:** Components communicate by publishing and reacting to events rather than calling each other directly — giving loose coupling, asynchrony, independent scaling, and **real-time** responsiveness (reacting within moments of something happening).

Part 0 — Concepts you need first

What is an "event"?

An **event** is a record that **something happened**. It's a statement of fact about the past.

The naming convention tells you a lot: events are named in the **past tense** — `OrderPlaced`, `PaymentReceived`, `UserRegistered`. This isn't cosmetic. It reflects that an event is a *notification of something already done*, not a request for something to be done.

Contrast with a **command**, which is an instruction, named imperatively — `PlaceOrder`, `ChargeCard`. A command can be refused. An event cannot; it already happened.

Synchronous vs. asynchronous

Synchronous — you make a request and **wait** for the response before doing anything else. Like a phone call: both parties are engaged for the duration.

Asynchronous — you send a message and **carry on immediately**, handling the response whenever it arrives. Like email: you send it and get on with your day.

The practical difference for a server: in a synchronous call, the calling **thread** (one line of execution inside the server, which handles one task at a time) is **blocked and useless** for the whole wait. In an asynchronous one, it's free immediately.

What is a message queue / event bus?

Middleware (software that sits between other programs to connect them) that accepts messages from senders and delivers them to receivers. **Kafka** (a distributed, durable event log) and **RabbitMQ** (a classic message broker) are the common ones. See System Design Fundamentals for queues in general.

The important properties:

- The **sender doesn't need to know who receives** — it publishes to the bus
- Messages are usually **persisted** (saved to disk), so if a receiver is down, the message waits
- **Many receivers** can consume the same message independently

What is publish-subscribe?

A messaging style where **publishers** send messages to a named channel (or "topic") without knowing who's listening, and **subscribers** register interest in a channel and receive everything published to it.

The essential property: **the publisher doesn't hold a list of subscribers.** Adding a new subscriber requires no change to the publisher — which is the entire source of EDA's flexibility.

What is "coupling"?

How much one component depends on knowledge of another.

Tight coupling: Service A calls `serviceB.doThing()`. A must know B exists, where it lives, its **API** (the operations it offers and how to call them), and its response format. Change B and you may break A.

Loose coupling: A publishes `ThingHappened`. It doesn't know whether anyone is listening. B may consume it, or five services may, or none. **A is unaffected either way.**

Part 1 — The problem

As applications grew, developers adopted **distributed systems** (many programs on different machines cooperating over a network) for **scalability** (handling more load by adding resources) and **resilience** (staying up when parts fail). But that shift brought challenges.

Managing complex interactions

A simple system has a small number of components interacting predictably. **Scale up and this becomes much harder.** A distributed system has a multitude of components that must interact and communicate. How do they communicate effectively? How do you ensure changes in one part are reflected in others?

The course's grounding example: consider how an email notification system works. When you receive an email saying a product has shipped, or that a blog you follow has a new post — that's a system efficiently handling complex interactions.

The call for asynchronous communication

In a synchronous system, a component makes a request and waits for the response before proceeding. This works for small, simple systems, but **quickly becomes a bottleneck** (the narrow point that limits everything) as the system grows.

> The question that makes it click: *what happens when you have thousands or even millions of components all waiting for responses?* Every waiting component is holding resources and doing nothing.

Asynchronous communication is often the answer — make a request, continue with other tasks, process the response when it arrives. Far more efficient, but **managing it effectively is a considerable challenge**.

The everyday example: why can you keep browsing an online store and adding items to your cart while previous requests are still being processed? Efficient asynchronous communication.

The demand for real-time responsiveness

Users expect immediate responses. Post a status update and you expect it visible to friends immediately. In online multiplayer games, any delay produces a poor experience.

Designing a system that manages complex interactions between components **and does so in real time** is no small feat.

Data consistency

With multiple components potentially reading and writing data **concurrently** (at the same time), how do you ensure consistency? If one component updates a piece of data, how do others become aware of the change?

The example: an online banking system. When you make a transaction it's critical your balance updates promptly and accurately — but what if another transaction is being processed on your account at the same moment?

Part 2 — The solution

EDA revolves around the **production, detection, consumption of, and reaction to events**.

An **event** is a change in state, or an update, produced by a component — the **event producer**. It's then detected and consumed by one or more **event consumers**.

> **The courier analogy:** a package (event) is sent by the sender (producer) and received by the recipient (consumer). The delivery service (event bus) takes care of delivering it. Scale it up to many senders and receivers and the core concept is unchanged.

Part 3 — The benefits

Decoupling

Event producers and consumers **aren't tied together and don't need to be aware of each other's existence**. This makes the system more flexible and adaptable to change.

The illustration that makes this concrete: a user uploads a photo to a social media app. That **single event** can trigger multiple actions:

- Notifying the user's followers
- Updating the user's profile
- Alerting the moderation team if the content violates policy

With EDA, **each of these is handled independently**. The system efficiently manages multiple tasks triggered by one event.

> The part worth pausing on: **six months later you want to add "generate a thumbnail."** With EDA you write a new consumer subscribing to `PhotoUploaded` and deploy it. **You never touch the upload code.** In a direct-call design you'd modify the upload service, retest it, and redeploy it — and repeat that for every new reaction, forever. That difference in change cost is the pattern's real value.

Asynchrony

The producer simply produces the event and **carries on with its operations without waiting** for consumers to process it. This **non-blocking** operation (the caller never sits idle waiting) greatly enhances the system's overall responsiveness and **throughput** (how much work gets done per second).

Scalability

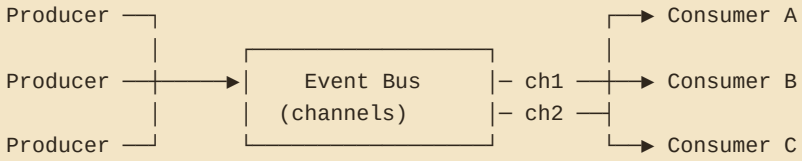
Since producers and consumers are decoupled, they can be **scaled independently** based on their workload. A surge of events from producers? Increase the number of consumer **instances** (running copies) to handle the **load** (volume of incoming work) — and vice versa.

Real-time processing

As soon as an event is produced, it can be **immediately handled** by the consumer. This is particularly useful where instant action is required — **fraud detection in banking transactions**, or real-time **analytics** (computing statistics and reports from data).

Part 4 — The components

Component	Role
Event Producers	Create events. They define what an event is, encapsulate the relevant data, and send it to the Event Bus
Event Bus (message queue)	The backbone. Receives events from producers and transmits them to consumers. **This is what decouples them**
Event Channels	Pathways on the bus. They categorize events by nature, or by the kind of consumers they cater to, so each consumer listens only to the channel relevant to it
Event Consumers	Listen to events via the bus and react. The reaction could be starting a new process, or merely recording the event



Part 5 — Three structural variants

The choice depends on system requirements, complexity, and how much control you need over event handling.

Mediator EDA

A **central mediator** takes charge of event handling.

Producers send their events to the mediator. The mediator — **aware of all producers and consumers** — determines where to route them. It performs **event aggregation, filtering, and transformation** (combining several events into one, dropping irrelevant ones, and reshaping their content), then delivers the processed events to the appropriate consumers.

- **Advantage:** control. It allows complex routing and transformation logic, giving a high degree of control over event processing.
- **Disadvantage:** the mediator could become a **bottleneck**, limiting the system's scalability.

Broker EDA

No central mediator. Event producers send events directly to Event Channels on the Event Bus. Consumers subscribe to those channels and consume events directly. **The broker essentially *is* the event bus.**

- **Advantage:** scalability and simplicity. With no central mediator, it's easier to scale.
- **Disadvantage:** less control over event routing and processing.

Hybrid EDA

Combines elements of both. Adopted when a system requires a mix of **centralized control and high scalability** — for example, a mediator for a few complex flows and direct broker delivery for everything else.

Part 6 — Inner workings

Event generation and propagation

An event's life starts with a **change of state** — a user action, a sensor reading, an internal system update. An event is generated to encapsulate that change: essentially **a bundle of data representing a significant occurrence**.

Once generated it must be propagated. The **Event Bus** is the spinal cord, carrying events from producers to consumers via **Event Channels** — pathways that categorize and route events by type or nature, ensuring the right events reach the right consumers.

Routing

- **In Mediator EDA:** a central component decides routing based on event type and known consumers. It may also aggregate, filter, or transform events before sending them on.
- **In Broker EDA:** there's no central decision-maker. Producers place events directly into channels; consumers subscribe and consume. Simpler routing, but less controlled.

> Without routing, events would simply float aimlessly in the system. With it, events know precisely where to go.

Event processing and state change

Once an event reaches its consumer, it is processed. The consumer takes the event data and performs actions based on it, **leading to a change in system state** — anything from changing a value in a database to triggering a physical action in an **IoT** device (Internet of Things — a sensor or gadget connected to the network).

How consumers know when to act

Event consumers are **always listening** — monitoring their subscribed channels on the bus, waiting. When an event arrives, they act.

This listening mechanism is usually implemented using the **Observer** (objects register with a subject and get called back whenever it changes) or **Publish-Subscribe** patterns.

Real-time illustration — traffic management

Various sensors act as producers, generating an event for each vehicle detected. The events travel via the bus, through appropriate channels, to the consumers — traffic signals. Based on the events received (for example vehicle count), the signals **adjust their timing**, changing the system state: traffic flow.

Throughout, the system reacts in real time.

Event Sourcing and CQRS

Two significant related concepts:

Event Sourcing means changes to application state are stored as a **sequence of events**, not just as a **snapshot** (a single picture) of the current state. This allows:

- **Full auditability** (being able to prove what happened and when) — every change is recorded
- **Versioning**
- The ability to **"time-travel"** — see the system's state at any past point

> Worth grasping the shift: instead of storing "balance = £70," you store `Deposited £100`, `Withdrew £30`. The balance is **derived** by replaying those events. You've kept the history rather than just the conclusion — which is exactly how accounting ledgers have always worked.

CQRS involves **separating the read and write operations** of a system. This is especially useful in an EDA, as it lets you model your event handlers (write operations) separately from your views (read operations).

Part 7 — Worked example: traffic management (Java)

The event

```java
public class VehicleEvent {
    private final String vehicleId;
    private final String type;

    public VehicleEvent(String vehicleId, String type) {
        this.vehicleId = vehicleId;
        this.type = type;
    }

    public String getVehicleId() { return vehicleId; }
    public String getType()      { return type; }
}
```

This represents a vehicle's action — such as 'entering' or 'leaving' a zone.

Note the `final` fields and absence of setters — the event is immutable (it cannot be changed after creation; a **setter** is a method that changes a field, and `final` in Java forbids reassigning one). That's deliberate and correct. An event describes something that **already happened**; allowing a consumer to modify it would be meaningless (you can't change the past) and dangerous (other consumers would see the altered version).

The producer — a sensor

```java
public class Sensor implements EventProducer {
    private EventBus eventBus;

    public Sensor(EventBus eventBus) {
        this.eventBus = eventBus;
    }

    public void vehicleDetected(String vehicleId, String action) {
        VehicleEvent event = new VehicleEvent(vehicleId, action);
        eventBus.publish(event);
    }
}
```

Sensors at traffic signals act as producers. Every time a vehicle passes, the sensor generates a `VehicleEvent` and publishes it to the bus.

Notice what's absent: the sensor has no reference to any traffic light. It doesn't know consumers exist. It publishes and returns — **fire and forget**.

The bus and channels

```
public interface EventBus {
    void publish(VehicleEvent event);
    void addChannel(EventChannel channel);
}

public class SimpleEventBus implements EventBus {
    private List<EventChannel> channels = new ArrayList<>();

    @Override
    public void publish(VehicleEvent event) {
        for (EventChannel channel : channels) {
            channel.add(event);
        }
    }

    @Override
    public void addChannel(EventChannel channel) {
        channels.add(channel);
    }
}

public class SimpleEventChannel implements EventChannel {
    private List<EventConsumer> consumers = new ArrayList<>();

    @Override
    public void add(VehicleEvent event) {
        for (EventConsumer consumer : consumers) {
            consumer.onEvent(event);
        }
    }

    @Override
    public void addConsumer(EventConsumer consumer) {
        consumers.add(consumer);
    }
}
```

When an event is published to the bus, it is added to all the channels; each channel then passes it to all its consumers.

The consumer — a traffic light

```
public class TrafficLight implements EventConsumer {
    @Override
    public void onEvent(VehicleEvent event) {
        if ("entering".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " is entering. Adjusting traffic light timings.");
        } else if ("leaving".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " has left. Resetting traffic light timings.");
        }
    }
}
```

The traffic light reacts to vehicle events by adjusting its timings.

What this example simplifies — read before copying it

The structure is right; the mechanics are not production-shaped:

- **It's entirely synchronous and in-process** (everything runs inside one program). `publish` calls consumers **directly on the caller's thread**. The sensor doesn't return until every traffic light has finished processing — which destroys the asynchrony that is the pattern's main benefit. A real EDA uses Kafka or RabbitMQ, with consumers running in **separate processes** (separately running programs) and the producer returning immediately.
- **No persistence.** If the process dies, **in-flight** events (published but not yet processed) are gone. Real buses persist events so they survive restarts and can be replayed.
- **No error handling.** If one consumer throws, the loop breaks and later consumers never see the event.
- **The bus broadcasts to every channel** rather than routing by type, so channels don't actually categorize anything here.

Use it to understand the **roles** — producer, bus, channel, consumer — not the implementation.

Part 8 — Performance implications and considerations

Scalability

Each component is decoupled, so you can **scale them independently** based on demand. High volume of events but consumers can't keep up? Scale up consumers without affecting producers or channels. However, **ensuring balance to avoid overwhelming certain parts of the system requires careful consideration.**

Processing speed and latency

EDA commonly boosts performance because of its asynchronous nature — producers continue producing without waiting for consumers.

The flip side is latency (the delay between something happening and it being handled). As the system scales and event volume increases, **the time from an event being produced to it being processed can grow**, delaying responses. Overall speed is also affected by the choice of event channel — synchronous or asynchronous.

Error handling and message durability

What happens when an event fails to be processed? How do you ensure messages aren't lost in transit?

If a consumer fails to process an event, the system needs a mechanism to **retry processing, log the failure, or alert an operator**. Ensuring **message durability** might require **persistent storage** to hold events until they're successfully processed.

Note the trade-off the course flags: **adding these mechanisms complicates the system and impacts performance.**

System complexity

EDA undoubtedly introduces complexity. **Tracking the flow of events and debugging issues is challenging** due to the asynchronous and distributed nature of the system. **Special tools and practices** may be needed to monitor and debug effectively. Designing a system where every component understands the format and meaning of events is itself complex.

> ⚠️ **This is the cost beginners most underestimate.** In a synchronous system, a **stack trace** (the error report listing the chain of method calls that led to a failure) tells you the whole story

> of what called what. In EDA, causation is spread across services *and across time* — the event that caused today's bug was published twenty minutes ago by a service you weren't looking at. You need **correlation IDs** (a shared identifier threaded through every event in a causal chain) and **distributed tracing** (recording a request's journey across every service it touches) from day one, not bolted on after your first **incident** (a production problem).

Event schemas and backward compatibility

Since all communication is via events, **all components must understand the events' structure**. This means events should follow a **well-defined schema** (a formal description of the fields, their types and which are required).

If the schema changes — new fields added, existing ones removed or altered — you must ensure **backward compatibility** (new versions still work with things built for the old one) so consumers expecting the old schema don't break.

> Practical rule: **adding an optional field is safe; removing or renaming one is not.** Old consumers ignore fields they don't know about, but break when a field they depend on vanishes. **Schema registries** (services that store every schema version and reject incompatible changes) exist to enforce this.

Order of events

In a system where order of operations matters, EDA presents a challenge. Events are **asynchronous by nature and could be processed in any order**. If your application has dependencies between events, you'll need a way to maintain the required order — which introduces further complexity.

> Concretely: if `AccountCreated` and `AccountDeleted` arrive out of order, a consumer might delete an account that doesn't exist yet and then create one that should be gone. Kafka guarantees ordering *within a partition* (a topic is split into several **partitions**, each an ordered log that can be processed in parallel), which is why choosing a **partition key** (the value that decides which partition an event goes to, like account ID) matters.

Testing

How do you test an event-driven system? **Unit testing individual components isn't enough** (a **unit test** checks one small piece in isolation) — it won't cover the interactions between parts. You'll need comprehensive **integration and end-to-end tests** (tests that run several real components together, or the whole system from the user's side), which ensure the system works as a whole but bring added complexity in writing and maintaining them.

None of these should deter you — they're a guide to navigating the challenges. Every architecture has trade-offs; the trick is understanding them and making informed decisions.

Part 9 — Use cases

Real-time analytics

In an increasingly data-driven world, processing and analysing data in real time is a powerful advantage. As events are generated they can be **instantly consumed and processed**, giving real-time insights. Especially useful in **finance**, where real-time stock market data drives instantaneous trading decisions.

IoT systems

IoT devices generate a multitude of events — a sensor detecting an environmental change, a smart device being activated, a security system triggered. The **asynchronous nature of EDA is perfectly suited** to handling these. In a smart home, an increase in room temperature triggers an event consumed by the air conditioning system, which adjusts accordingly.

E-commerce systems

An e-commerce platform comprises subsystems like inventory, billing, and shipping. When a customer places an order it triggers a series of events: inventory verifies stock, billing processes payment, shipping handles delivery. With EDA these operations are carried out **asynchronously and efficiently**.

Microservices architecture

Services need to communicate. (**Microservices** are small, independently deployable services.) **Instead of direct communication — which leads to tightly coupled systems — services emit events that other services consume.** This keeps services loosely coupled and able to **evolve independently over time**.

Part 10 — System design example: a ride-sharing app

Designing something like Uber or Lyft with EDA. Various events drive the entire process.

1. **A rider requests a ride** → generates an event containing the necessary details such as pickup location and destination. The system **broadcasts** this event.
2. **Drivers in the vicinity** act as consumers of this event.
3. **A driver accepts the ride** → triggers another event, consumed by the rider's app, informing them of the match.
4. **During the ride**, the driver's app continuously sends events about the **current location**. The rider's app consumes these and **updates the display in real time**.
5. **The ride ends** → the driver's app sends a **ride completion event**.
6. **The billing service** consumes it, calculates the fare, and **triggers a payment event**.

This showcases EDA handling complex workflows while promoting real-time, asynchronous processing and loosely coupled design.

> Notice how naturally this maps: a ride genuinely *is* a sequence of things that happened. When your domain is already event-shaped, EDA stops feeling like a technical choice and starts feeling like the obvious model.

Part 11 — Common beginner misconceptions

"Events are just asynchronous function calls." No. A function call has one specific target and expects a return. An event has **zero or many** targets, unknown to the publisher, and expects nothing.

"EDA means no more direct calls anywhere." No. When you need an immediate answer — "is this user authorized?" — a synchronous call is right. EDA is for notification and reaction, not for queries.

"The event bus makes my system reliable." It makes it *decoupled*. Reliability requires persistence, retries, **dead-letter queues** (holding areas for messages that keep failing), and **idempotent** consumers (handling the same event twice has the same effect as once) on top.

"If I publish an event, it definitely gets processed." Only with **durable** messaging (stored safely until processed) and **acknowledgements** (the consumer confirming to the broker that it finished a

message). Fire-and-forget over an in-memory bus loses events on restart.

"Consumers can assume they see each event exactly once." Most systems guarantee *at-least-once* delivery (every message arrives, but possibly more than once), meaning **duplicates happen**. Consumers should be idempotent. *(See Retry.)*

"Ordering is guaranteed." Generally not across the whole system — only within a partition or channel, if your **broker** (the messaging server) offers even that.

Part 12 — Check your understanding

1. Why are events named in the past tense?
2. What's the change-cost argument for EDA, using the photo upload example?
3. What's the difference between Mediator and Broker EDA, and what does each trade away?
4. Why is debugging harder in EDA, and what two tools address it?
5. Which schema changes are safe and which break consumers?
6. What's wrong with the example's `publish` method, relative to real EDA?

(Answers: 1 — an event records something that already happened and cannot be refused, unlike a command; 2 — adding a new reaction means writing a new consumer without ever modifying the upload service, whereas direct calls require changing and redeploying the producer each time; 3 — Mediator centralizes routing and transformation, gaining control but risking a bottleneck, while Broker publishes straight to channels, gaining scalability but losing routing control; 4 — causation is spread across services and time so there's no single stack trace, addressed by correlation IDs and distributed tracing; 5 — adding optional fields is safe, removing or renaming is not; 6 — it calls consumers synchronously on the producer's thread, so the producer blocks until all consumers finish, eliminating the asynchrony that is the point.)

Part 13 — Interview soundbite (30 seconds)

> "Event-driven architecture has components communicate by publishing events rather than calling each other directly. A producer publishes a state change to an event bus, and any number of consumers subscribe to the channels they care about. The producer doesn't know who's listening, which is the whole point — when a user uploads a photo, that one event can trigger follower notifications, a profile update, and content moderation, and six months later you add thumbnail generation by writing a new consumer without ever touching the upload code. You also get asynchrony, so producers don't block, and independent scaling of producers and consumers. There are three variants: broker, where producers publish straight to channels — simple and scalable; mediator, where a central component routes, filters, and transforms — more control but a potential bottleneck; and hybrid. Ride-sharing is a natural example, with ride requests, driver acceptance, live location updates, and completion all as events. The costs are real: ordering isn't guaranteed across the system, you have to version event schemas carefully because removing a field breaks old consumers, delivery is usually at-least-once so consumers must be idempotent, and debugging is much harder because causation spans services and time — so you need correlation IDs and distributed tracing from the start. It pairs naturally with event sourcing, where state is stored as the sequence of events rather than a snapshot, and with CQRS."

Part 14 — Implementation

A. Runnable demo

Full source: `demos/EventDrivenDemo.java`

```
cd demos && ./gradlew runCh11
```

Failure isolation — one consumer throwing must not affect the others:

```
for (Subscriber s : subs) {
    Runnable task = () -> {
        try { s.handler().accept(e); }
        catch (Exception ex) {
            System.out.printf("      [%s] FAILED: %s (isolated)%n", s.name(), ex.getMessage());
        }
    };
    if (async) pool.submit(task); else task.run();
}
```

Idempotent consumer, using a set of seen event IDs:

```
void handle(Event e) {
    String eventId = e.payload().get("eventId");
    if (!processed.add(eventId)) return;      // add() is false if already present
    chargesApplied++;
}
```

B. What you should see

```
=== 1. SYNCHRONOUS bus: producer waits for every consumer ===
  producer blocked for 114ms  <- the sum of all consumers

=== 2. ASYNCHRONOUS bus: producer returns immediately ===
  producer returned in 0ms  <- did NOT wait
      [moderation] FAILED: moderation service unavailable (isolated)

=== 4. Add a NEW consumer - producer code untouched ===
    [bus] publishing PhotoUploaded (cid=cid-3) to 4 subscriber(s)
  >> 4 reactions now. The upload service never changed.

=== 5. At-least-once delivery -> idempotent consumers ===
      [billing]    charge applied (total=1)
      [billing]    duplicate evt-abc ignored
      [billing]    duplicate evt-abc ignored
  charges actually applied: 1 (must be 1)
```

114ms → 0ms is the value of asynchrony as a measured number. And section 4 is the whole architectural argument: the subscriber count went from 3 to 4 and **the producer's code was never edited**.

C. Production — Kafka

(In this config, `bootstrap-servers` are the first Kafka machines a client contacts; `acks: all` makes the producer wait until every **replica** — a copy of the data on another broker — has the message; an **offset** is a consumer's position in a partition, and **committing** it records how far the consumer has read.)

```yaml
spring:
  kafka:
    bootstrap-servers: kafka-1:9092,kafka-2:9092,kafka-3:9092
    producer:
      acks: all                          # wait for all replicas - no silent data loss
      retries: 3
      properties:
        enable.idempotence: true         # broker de-duplicates producer retries
        max.in.flight.requests.per.connection: 5
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      enable-auto-commit: false          # commit only AFTER successful processing
      isolation-level: read_committed
```

Producer — the partition key is the important decision (`UUID` generates a unique ID; `MDC` is the logging library's per-request context, used here to carry the correlation ID):

```java
@Service
public class PhotoEventPublisher {
    private final KafkaTemplate<String, PhotoUploaded> kafka;

    public void publish(String photoId, String userId) {
        PhotoUploaded event = new PhotoUploaded(
                UUID.randomUUID().toString(),      // eventId, for consumer de-duplication
                MDC.get("correlationId"),          // trace the causal chain
                photoId, userId, Instant.now());

        // Key = userId -> all events for one user land on one partition,
        // and Kafka guarantees ordering WITHIN a partition.
        kafka.send("photo-uploaded", userId, event);
    }
}
```

Consumer — manual acknowledgement plus idempotency:

```java
@KafkaListener(topics = "photo-uploaded", groupId = "thumbnail-service")
public void onPhotoUploaded(PhotoUploaded event, Acknowledgment ack) {
    if (processedEvents.existsById(event.eventId())) {   // at-least-once => duplicates happen
        ack.acknowledge();
        return;
    }
    try {
        thumbnailService.generate(event.photoId());
        processedEvents.save(new ProcessedEvent(event.eventId(), Instant.now()));
        ack.acknowledge();                                // commit only on success
    } catch (TransientException e) {
        // Do NOT ack - Kafka redelivers
        throw e;
    }
}
```

Dead-letter topic (DLT) — where messages go after retries are exhausted:

```java
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template,
            (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));
    return new DefaultErrorHandler(recoverer,
            new ExponentialBackOff(1000L, 2.0));   // 1s, 2s, 4s...
}
```

Schema evolution — register schemas so a **breaking change** (one that would make existing consumers fail) is rejected at build time, not discovered in production:

```yaml
spring.kafka.properties:
  schema.registry.url: http://schema-registry:8081
  value.subject.name.strategy: io.confluent.kafka.serializers.subject.TopicNameStrategy
```

Compatibility mode `BACKWARD` means **new consumers can read old events**. Adding an optional field passes; removing a field fails the build.

D. Implementation pitfalls

1. Auto-commit enabled. With `enable-auto-commit: true`, Kafka commits the offset on a timer — possibly *before* your handler finishes. Crash mid-processing and the message is **silently lost**. Always commit manually after success.

2. Consumers that aren't idempotent. At-least-once delivery is the norm, so duplicates are guaranteed, not hypothetical. Store processed event IDs.

3. No partition key, then expecting ordering. Kafka orders **within a partition only**. Without a key, events for the same **entity** (one business object, such as a specific account) scatter across partitions and arrive out of order. Key by entity ID.

4. Publishing before committing the database transaction. Classic bug: you write to the DB, then publish, and crash in between — the state changed but nobody was told. Use the **Transactional Outbox** (save the event in an outbox table in the same database transaction, then publish it separately) *(see Saga Part 13D)*.

5. Events carrying too much or too little. Too little and every consumer must call back to the producer, recreating coupling. Too much and the event becomes a schema everyone depends on. Carry **identifiers plus the facts that actually changed**.

6. Removing or renaming event fields. Old consumers break. Additive changes only; use a schema registry to enforce it.

7. No correlation ID. Without one you cannot reconstruct what caused what. Generate at the edge, propagate through every event, log on every line.

8. No dead-letter queue. A permanently-failing message blocks its partition forever, stalling every message behind it. Always configure a DLT with alerting.

Related patterns

- **CQRS** — separate read/write models, commonly built on EDA
- **Saga** — choreography sagas are EDA applied to distributed transactions
- **Retry** — requeueing and dead-letter queues are retry in an async context
- **Bulkhead** — a queue between components is a natural isolation boundary